Week 11
Contextual Bandits and Off-Policy Evaluation

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4
1.1 Two running examples . . . 4
1.2 Anchoring to what you already know . . . 4

2 Mathematical formulation 5
2.1 UCB1 and Hoeffding . . . 5
2.2 Thompson Sampling . . . 6
2.3 Contextual bandits and LinUCB . . . 6
2.4 MOSS: removing the spurious $\log T$ . . . 6
2.5 LinTS and neural extensions . . . 7
2.6 Off-Policy Evaluation: setup . . . 7
2.7 MIPS: OPE for large action spaces . . . 8
2.8 Effective sample size . . . 8
2.9 Visual intuition . . . 9

3 Worked numerical examples and pitfalls 10
3.1 Example: ESS collapse . . . 10
3.2 Example: clipping changes the answer . . . 10
3.3 Example: positivity violation . . . 10
3.4 Pitfalls compendium . . . 10

4 Algorithms 12

5 Implementation walkthrough 14

6 Production pipeline: synthetic insurance in detail 16
6.1 Data contract . . . 16
6.2 Allocation decision . . . 16
6.3 Logging . . . 16
6.4 Delayed outcome . . . 17
6.5 Audit . . . 17
6.6 Kill-switch . . . 17

7 Empirical results 18

8 Safety angle 21

9 Matching application 22

10 Exercises 23

11 Further reading 24

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Derive the UCB1 regret bound sketch from Hoeffding and Lai–Robbins, and contrast it with KL-UCB, MOSS, and Thompson Sampling.
2. Show that the LinUCB posterior mean is the closed-form ridge solution $\hat{\theta}_a = A_a^{-1} b_a$ with $A_a = \lambda I + \sum_t x_t x_t^\top$.
3. Prove that Inverse Propensity Scoring (IPS) is unbiased, derive its variance, and explain why Self-Normalised IPS (SNIPS) is biased but often dominant in mean squared error.
4. Derive the Doubly Robust estimator and prove its double-robustness: consistent if *either* the propensity *or* the reward model is correct.
5. Build a synthetic synthetic insurance lead–broker contextual bandit (6 brokers, 8 lead features), collect ε-greedy logs, and run SNIPS, Direct, DR, and SWITCH-DR against three counterfactual policies.
6. Read off a bias/variance trade-off plot and pick τ for SWITCH.
7. State the four production gotchas: logging propensities, overlap (positivity) failure, delayed feedback bias, and propensity overfitting.

1 Motivation and intuition

A *bandit* is a Markov Decision Process collapsed to one step: at every round you observe a context x_t, pick an arm $a_t \in \{1, \ldots, K\}$, and receive a noisy reward r_t. There is no transition; the next round's context is independent of your action.

The puzzle is the same as in MDPs — balance *exploration* (try arms whose value you do not know) with *exploitation* (use the best arm you have seen). What is gone is the credit-assignment chain. What is gained is **a fully solvable, tightly characterised problem** where every algorithmic trick (UCB, posterior sampling, importance sampling, doubly robust estimation) has a clean derivation and a sharp regret bound. Master this and the deeper RL material reduces to "do this, but with a target network and a replay buffer".

Why bother with such a small problem? Because almost every applied ML system is at heart a bandit once the deep-learning fog clears: pick this ad / pick this thumbnail / pick this broker.

1.1 Two running examples

synthetic insurance lead-broker matching. A motor-insurance comparator routes each incoming *lead* (a quote form) to one of K brokers. The reward is *bind*: did the broker convert the lead into a sold policy? Features include postcode band, vehicle power, prior claims, channel (direct vs. aggregator), age band. The platform must (i) maximise binds, (ii) keep brokers happy by sending them leads that match their book of business, and (iii) audit the policy offline before pushing changes to production. *This is a contextual bandit with delayed, partially observed feedback.*

ATS candidate–job ranking. (Treated in depth in Week 12.) A recruiter platform must rank candidates for a requisition. The reward is hire. Position bias makes naive click-modelling actively misleading. The re-ranker is a contextual bandit on top of a two-tower retriever.

1.2 Anchoring to what you already know

If you know logistic regression, you know LinUCB — it is logistic / ridge regression with a confidence ellipsoid bolted on. If you know dropout as a Bayesian approximation, you know Thompson Sampling — it is posterior sampling, but cheaper. If you know cross-validation, you know off-policy evaluation — it is the same idea: estimate held-out performance, but with an importance-weight correction because the held-out set was generated by a different policy.

Common pitfall

Bandit literature is split between the *stochastic* setting (rewards i.i.d. given arm) and the *adversarial* setting (rewards chosen by an adversary). Almost everything in this chapter is stochastic. Production matching is somewhere in between: brokers and recruiters are slow-moving adversaries (incentive-compatibility) on top of i.i.d. leads. We address that in Week 12; for now, assume i.i.d.

2 Mathematical formulation

Definition 2.0.1 (Stochastic K-armed bandit)**.** A tuple $(\mathcal{A}, P)$ with $|\mathcal{A}| = K$ and $P = (P_a)_{a\in\mathcal{A}}$ a family of reward distributions on $[0,1]$. At each round $t = 1, 2, \dots$ the learner picks $A_t \in \mathcal{A}$ and observes $R_t \sim P_{A_t}$ independently of the past given A_t. Let $\mu_a = \mathbb{E}[R \mid a]$ and $\mu^\star = \max_a \mu_a$. The *regret* is

$$\mathrm{Reg}(T) \triangleq T\mu^\star - \mathbb{E}\Big[\sum_{t=1}^{T} R_t\Big] = \sum_a \Delta_a \,\mathbb{E}[N_a(T)],$$

where $\Delta_a = \mu^\star - \mu_a$ and $N_a(T) = \sum_t \mathbb{1}\{A_t = a\}$.

Theorem 2.0.2 (Lai–Robbins lower bound, 1985)**.** *For any* consistent *policy and any Bernoulli bandit,* $\liminf_{T\to\infty} \mathrm{Reg}(T)/\log T \geq \sum_{a:\Delta_a>0} \Delta_a/\mathrm{KL}(\mu_a, \mu^\star)$.

The lower bound says: regret must grow at least like $\log T$, and the "hardness" constant is governed by KL divergences. UCB1 achieves the $\log T$ rate but with a constant $1/\Delta_a^2$ (Hoeffding-loose); KL-UCB achieves the KL constant.

2.1 UCB1 and Hoeffding

Theorem 2.1.1 (Hoeffding's inequality)**.** *Let $X_1, \dots, X_n$ be i.i.d. in $[0,1]$ with mean μ. Then for any $\delta > 0$,*

$$\mathbb{P}\Big(|\bar{X}_n - \mu| > \sqrt{\tfrac{\log(2/\delta)}{2n}}\Big) \leq \delta.$$

UCB1 (Auer, Cesa-Bianchi, Fischer 2002) plays

$$A_t = \arg\max_a \Big[\hat{\mu}_a(t-1) + \sqrt{\tfrac{2\log t}{N_a(t-1)}}\Big].$$

The bonus is the Hoeffding radius with $\delta = 2t^{-4}$ — shrinking fast enough that the bad-confidence event is summable.

Theorem 2.1.2 (UCB1 regret, sketch)**.** $\mathbb{E}[\mathrm{Reg}(T)] \leq \sum_{a:\Delta_a>0} \big(\frac{8\log T}{\Delta_a} + (1+\pi^2/3)\Delta_a\big)$.

Sketch. A sub-optimal arm a is played at round t only if $\hat{\mu}_a + \sqrt{2\log t/N_a} \geq \hat{\mu}^\star + \sqrt{2\log t/N^\star}$. By Hoeffding, $\hat{\mu}^\star \geq \mu^\star - \sqrt{2\log t/N^\star}$ with probability $\geq 1 - t^{-4}$, and similarly $\hat{\mu}_a \leq \mu_a + \sqrt{2\log t/N_a}$. So either a low-probability deviation occurred, or $\mu_a + 2\sqrt{2\log t/N_a} \geq \mu^\star$, i.e. $N_a \geq 8\log t/\Delta_a^2$. Hence $\mathbb{E}[N_a(T)] \leq 8\log T/\Delta_a^2 + \sum_t 2t^{-4}$. Multiplying by Δ_a and summing the bounded tail gives the bound. □

Remark. The slack between Lai–Robbins (Δ_a/KL) and UCB1 ($\Delta_a/\Delta_a^2 = 1/\Delta_a$) comes from Hoeffding being loose for Bernoulli rewards near 0 or 1. KL-UCB closes this slack by replacing Hoeffding with the *KL* upper confidence bound:

$$\mathrm{UB}_a(t) = \max\{q \in [\hat{\mu}_a, 1] : N_a \cdot \mathrm{KL}(\hat{\mu}_a, q) \leq \log t + c\log\log t\}.$$

2.2 Thompson Sampling

Definition 2.2.1 (Thompson Sampling, Bernoulli)**.** Maintain a Beta(α_a, β_a) posterior per arm. Initialise $\alpha_a = \beta_a = 1$. At round t: sample $\tilde{\mu}_a \sim \text{Beta}(\alpha_a, \beta_a)$, play $A_t = \arg\max_a \tilde{\mu}_a$, observe $R_t \in \{0, 1\}$, update $\alpha_{A_t} \mathrel{+}= R_t$, $\beta_{A_t} \mathrel{+}= 1 - R_t$.

Theorem 2.2.2 (Agrawal & Goyal 2012/2013)**.** *Bernoulli TS satisfies* $\mathbb{E}[\text{Reg}(T)] = O(\sum_a (\log T)/\Delta_a)$*, matching Lai–Robbins up to constants. The proof reduces to bounding the time the posterior on a sub-optimal arm dominates the optimal arm's posterior, using anti-concentration of the Beta.*

Intuition. TS is "OFU by sampling". UCB picks the upper confidence bound; TS picks a draw from the posterior. Both are aggressive enough to explore, but TS is *randomised*, which makes it composable with batched updates, delayed feedback, and ensembles. In every industry A/B test we have seen, TS matches or beats UCB.

2.3 Contextual bandits and LinUCB

Definition 2.3.1 (Linear contextual bandit)**.** At each round t: observe $x_t \in \mathbb{R}^d$ with $\|x_t\|_2 \le 1$; pick $A_t \in [K]$; observe $R_t = \langle x_t, \theta_{A_t} \rangle + \eta_t$, with η_t conditionally σ-sub-Gaussian. Hidden parameters θ_a are bounded ($\|\theta_a\|_2 \le S$).

Theorem 2.3.2 (LinUCB posterior mean is ridge regression)**.** *Given logged data* $(x_s, A_s, R_s)_{s \le t}$ *and ridge penalty* λ*, the posterior mean for arm* a *under a Gaussian prior* $\theta_a \sim \mathcal{N}(0, \lambda^{-1} I)$ *and Gaussian likelihood is*

$$\hat{\theta}_a \;=\; A_a^{-1} b_a, \qquad A_a = \lambda I + \sum_{s : A_s = a} x_s x_s^\top, \quad b_a = \sum_{s : A_s = a} R_s x_s.$$

Proof. The Gaussian posterior on θ_a is $\mathcal{N}(\hat{\theta}_a, \sigma^2 A_a^{-1})$ with $\hat{\theta}_a = (\sigma^2 \lambda I + \sum_s x_s x_s^\top)^{-1} \sum_s R_s x_s$. With σ^2 absorbed into λ this is exactly $A_a^{-1} b_a$. □ □

LinUCB acts by

$$A_t = \arg\max_a \langle x_t, \hat{\theta}_a \rangle + \alpha \sqrt{x_t^\top A_a^{-1} x_t},$$

where the second term is the width of a $1 - \delta$ confidence ellipsoid: $\|\theta_a - \hat{\theta}_a\|_{A_a} \le \alpha$ with $\alpha = \sigma \sqrt{d \log((1 + t/\lambda)/\delta)} + \sqrt{\lambda}\, S$ (Abbasi-Yadkori, Pál, Szepesvári 2011).

The classical Li et al. (WWW 2010) constant was empirical; the tight α above is the self-normalised concentration of Abbasi-Yadkori et al.

Theorem 2.3.3 (LinUCB regret)**.** $\mathbb{E}[\text{Reg}(T)] = \tilde{O}(d\sqrt{T})$.

Sketch. The per-step regret is bounded by twice the confidence width on the chosen arm: $r_t \le 2\alpha_t \sqrt{x_t^\top A_{A_t}^{-1} x_t}$. Summing: $\sum_t r_t^2 \le 4\alpha_T^2 \sum_t x_t^\top A_{A_t}^{-1} x_t \le 4\alpha_T^2 \cdot d \log(1 + T/(\lambda d))$ by the elliptical-potential lemma. Cauchy–Schwarz: $\sum_t r_t \le \sqrt{T} \sqrt{\sum_t r_t^2} = \tilde{O}(d\sqrt{T})$. □ □

2.4 MOSS: removing the spurious $\log T$

UCB1 has a $\log T$ term in the bonus that produces a $\log T$ factor in the worst-case (minimax) regret. *MOSS* (Audibert & Bubeck 2009) replaces the bonus with $\sqrt{\log_+(T/(Kn_a))/n_a}$, where $\log_+(z) = \max(0, \log z)$. The minimax regret becomes $O(\sqrt{KT})$ — this is the minimax-optimal rate for K-armed bandits with horizon T.

Intuition. The $\log T$ in UCB1 is honest: it is the price of *instance-optimal* regret on easy instances (Δ_a large). MOSS sacrifices a bit of instance optimality near the boundary for the cleaner minimax rate. In practice, KL-UCB and TS dominate both on Bernoulli reward landscapes; MOSS is the theoretically correct reference when reward gaps are unknown a priori.

2.5 LinTS and neural extensions

LinTS (Agrawal & Goyal 2013) replaces the UCB radius with one Gaussian draw per arm: $\tilde{\theta}_a \sim \mathcal{N}(\hat{\theta}_a, v^2 A_a^{-1})$, then act greedily. Frequentist regret $\tilde{O}(d^{3/2}\sqrt{T})$, empirically as good as LinUCB and easier to tune (one variance scalar instead of a deviation constant).

NeuralUCB, NeuralTS, bootstrapped TS. When the reward is non-linear in features (CV embeddings vs. JD embeddings), the linear posterior is inappropriate. The pragmatic stack is: a frozen text encoder produces features $\phi(x) \in \mathbb{R}^d$, then a *Bayesian linear head* per arm runs LinUCB or LinTS. The encoder is shared (no per-arm capacity blow-up), the head is calibrated. This is what we will deploy in Week 12.

2.6 Off-Policy Evaluation: setup

You have a log $\mathcal{D} = \{(x_i, a_i, r_i, p_i)\}_{i=1}^n$ with $p_i = \pi_b(a_i \mid x_i)$ the *recorded propensity* of the behaviour (logging) policy. You want to estimate

$$V(\pi) = \mathbb{E}_{x\sim D_x}\, \mathbb{E}_{a\sim\pi(\cdot|x)}\, \mathbb{E}_{r\sim P(\cdot|x,a)}[r]$$

for a counterfactual target π *without* deploying π.

Definition 2.6.1 (IPS estimator, Horvitz–Thompson)**.**

$$\hat{V}_{\mathrm{IPS}}(\pi) = \frac{1}{n}\sum_{i=1}^{n} \frac{\pi(a_i \mid x_i)}{\pi_b(a_i \mid x_i)} r_i.$$

Theorem 2.6.2 (IPS is unbiased under positivity)**.** *Assume correct logged propensities, a fixed evaluation policy independent of the evaluation sample, the same context distribution and conditional reward mechanism at logging and evaluation, and* positivity*: $\pi_b(a \mid x) > 0$ whenever $\pi(a \mid x) > 0$. Condition on all outcome-relevant action-selection variables (or randomise actions). Then $\mathbb{E}[\hat{V}_{\mathrm{IPS}}(\pi)] = V(\pi)$.*

Proof. For a single sample,

$$\begin{aligned}
\mathbb{E}\Big[\tfrac{\pi(a|x)}{\pi_b(a|x)} r\Big] &= \mathbb{E}_x \mathbb{E}_{a\sim\pi_b(\cdot|x)} \mathbb{E}_{r\sim P(\cdot|x,a)}\Big[\tfrac{\pi(a|x)}{\pi_b(a|x)} r\Big] \\
&= \mathbb{E}_x \sum_a \pi_b(a|x) \cdot \tfrac{\pi(a|x)}{\pi_b(a|x)} \cdot \mathbb{E}_r[r|x,a] \\
&= \mathbb{E}_x \sum_a \pi(a|x)\, q(x,a) = V(\pi).
\end{aligned}$$

□ □

Theorem 2.6.3 (IPS variance)**.**

$$\mathrm{Var}[\hat{V}_{\mathrm{IPS}}(\pi)] = \frac{1}{n}\Big(\mathbb{E}\Big[\tfrac{\pi(a|x)^2}{\pi_b(a|x)^2} r^2\Big] - V(\pi)^2\Big),$$

For i.i.d. logs and finite second moments, this is governed by $\mathbb{E}_b[\rho^2 r^2]$, not $\mathbb{E}_b[\rho]$, which equals 1 under support. Peaky targets can produce large variance when logger probability is small on their preferred actions; a deterministic target with good overlap need not.

Definition 2.6.4 (SNIPS)**.**

$$\hat{V}_{\mathrm{SNIPS}}(\pi) = \frac{\sum_i w_i r_i}{\sum_i w_i}, \quad w_i = \tfrac{\pi(a_i|x_i)}{\pi_b(a_i|x_i)}.$$

SNIPS is generally biased in finite samples and consistent under support and integrability. An $O(1/n)$ bias expansion needs suitable moment conditions. Its bias sign and variance ordering relative to IPS are not universal; a zero denominator is undefined.

Definition 2.6.5 (Direct method)**.** Fit a reward model $\hat{q}(x, a) \approx \mathbb{E}[r \mid x, a]$ on the log, then

$$\hat{V}_{\mathrm{DM}}(\pi) = \frac{1}{n} \sum_i \sum_a \pi(a \mid x_i)\, \hat{q}(x_i, a).$$

Low variance, biased by model misspecification.

Theorem 2.6.6 (Doubly Robust, Dudík–Langford–Li 2011)**.**

$$\hat{V}_{\mathrm{DR}}(\pi) = \frac{1}{n} \sum_i \Big[\sum_a \pi(a|x_i)\hat{q}(x_i, a) + w_i(r_i - \hat{q}(x_i, a_i)) \Big].$$

With support, a fixed target policy, and nuisance predictions independent of each evaluation row, correctness of either the propensity or conditional reward model gives unbiasedness. Use sample splitting or cross-fitting for learned nuisance models; same-row fitting does not automatically retain the exact finite-sample result.

Proof. **Case 1: π_b correct.** Take expectation conditional on x: $\mathbb{E}[w(r - \hat{q}(x, a)) \mid x] = \sum_a \pi(a|x)(q(x, a) - \hat{q}(x, a))$. Adding $\sum_a \pi(a|x)\hat{q}(x, a)$ gives $\sum_a \pi(a|x)q(x, a) = V(\pi|x)$.

Case 2: $\hat{q} = q$. Then $r - \hat{q}(x, a)$ is mean-zero given x, a, so the second term is mean-zero regardless of π_b. The first term equals $V(\pi|x)$. □ □

Remark (DR variance). $\mathrm{Var}[\hat{V}_{\mathrm{DR}}] - \mathrm{Var}[\hat{V}_{\mathrm{IPS}}]$ is governed by weighted reward residuals and variation in the model-based conditional value. A useful reward model often reduces variance, but arbitrary approximation changes do not give a universal monotonic comparison. In the best case (perfect $\hat{q}$), DR still includes weighted outcome noise. It reduces to direct-method variance only when reward residuals have zero conditional variance; knowing a Bernoulli mean does not remove Bernoulli noise.

Definition 2.6.7 (SWITCH-DR, Wang–Agarwal–Dudík 2017)**.** Pick a threshold τ. For each sample with weight $w_i \le \tau$ use DR; for $w_i > \tau$ fall back to the direct method. Trades a small bias for a big variance cut.

2.7 MIPS: OPE for large action spaces

When $K \gg n$ (think: $K = 6000$ jobs, $n = 50000$ logs) IPS variance is fatal because most (x, a) pairs are never observed. *MIPS* (Marginalised IPS; Saito & Joachims, ICML 2022) embeds the action into $e \in \mathbb{R}^{d_e}$ and replaces the per-action propensity with the marginal action-embedding propensity:

$$\hat{V}_{\mathrm{MIPS}}(\pi) = \frac{1}{n} \sum_i \frac{\sum_{a'} \pi(a' \mid x_i) p(e_i \mid x_i, a')}{\sum_{a'} \pi_b(a' \mid x_i) p(e_i \mid x_i, a')} r_i.$$

Theorem 2.7.1 (MIPS identification conditions)**.** *Under embedding sufficiency ($r \perp\!\!\!\perp a \mid x, e$), correct marginal embedding weights, and target-to-logger embedding support, MIPS is unbiased. Marginalisation can reduce variance, but no general $O(d_e/n)$ versus $O(K/n)$ comparison follows from embedding dimension alone. See Saito and Joachims (2022), assumptions 3.1–3.2.*

2.8 Effective sample size

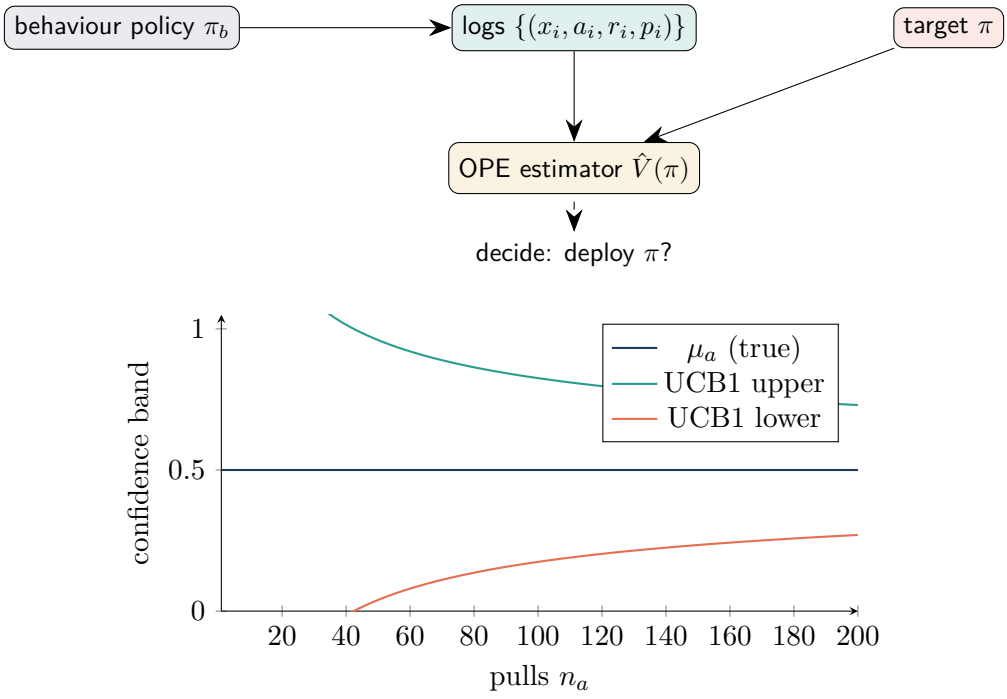
A diagnostic for whether OPE estimates are trustworthy:

$$\mathrm{ESS} \triangleq \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

$\mathrm{ESS} \in [1, n]$. $\mathrm{ESS} \ll n$ indicates that a tiny fraction of log entries carries most of the weight — the estimator is effectively relying on a handful of samples. Common production thresholds: refuse to report an OPE number if $\mathrm{ESS} < 100$ or $\mathrm{ESS}/n < 0.05$.

Intuition. ESS is the bandit analogue of the rank of a design matrix in linear regression: it tells you how much information the data actually contains about the quantity you are estimating. A 1-million-row log with ESS $= 50$ is a 50-row dataset wearing a costume.

2.9 Visual intuition



Common pitfall

Positivity is not a luxury. If $\pi_b(a \mid x) = 0$ for some (x, a) the target might assign, IPS and DR are biased and SNIPS is silently wrong. Use ε-greedy or stochastic propensity floors at logging time.

3 Worked numerical examples and pitfalls

3.1 Example: ESS collapse

Suppose you log $n = 10\,000$ samples from a $\varepsilon = 0.05$-greedy policy on a $K = 100$-arm bandit. A target policy is the greedy oracle. For samples where the logger picked the same action as the oracle, the importance weight is $w = 1/(1-\varepsilon+\varepsilon/K) \approx 1.05$. For all other samples, $w = 1/(\varepsilon/K) = 2000$. Suppose the logger picks the oracle's action 5% of the time; then 500 samples have $w \approx 1$ and 9500 samples have $w = 2000$. The effective-sample-size diagnostic is dominated by background mass; ESS within the target's support is the more honest number.

3.2 Example: clipping changes the answer

A common production trick is to clip importance weights at $w_{\max}$. This bounds variance but introduces bias. For a policy where 1% of samples have $w = 1000$ and the rest have $w = 1$, the true mean is ~ 10 but clipping at $w_{\max} = 10$ pulls every large weight to 10, giving an estimate of ~ 1.

Practical recipe. Always report the fraction of clipped samples alongside the estimate. If $> 5\%$ of samples are clipped, the estimate is unreliable; use SWITCH-DR or MIPS instead.

3.3 Example: positivity violation

Suppose your logger is deterministic on lead category "commercial motor" (always sends to broker 3). A target policy wants to send commercial-motor leads to broker 4. The importance weight for those samples is zero; IPS returns zero on them; the estimate is biased downward. The fix is structural: either randomise the logger (ε-greedy with $\varepsilon > 0$) or restrict the target to actions the logger actually explores.

3.4 Pitfalls compendium

Common pitfall

Confusing π_b and target π. The propensity is the behaviour policy's probability of taking the logged action, not the target's. The sanity-check is that the unweighted average of importance weights converges to 1.

Common pitfall

Counting clipped samples as un-clipped. A common production sin: "we clipped 30% of weights and the estimator looks fine". It is not fine; clipping introduces an unbounded bias.

Common pitfall

OPE across a policy version boundary. Logs from one policy version are not directly usable for evaluating against logs from another version. Either restrict the OPE window to a single version or weight by version occupancy.

4 Algorithms

Algorithm 1: LinUCB (disjoint arms)

Input: K, d, exploration radius α, ridge λ.
Init: $A_a \leftarrow \lambda I_d$, $b_a \leftarrow 0$ for all a.
for $t = 1, 2, \dots$ **do**
  observe x_t
  for $a \in [K]$ **do**
    $\hat{\theta}_a \leftarrow A_a^{-1} b_a$
    $p_a \leftarrow \hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}$
  pick $A_t = \arg\max_a p_a$
  observe r_t
  $A_{A_t} \leftarrow A_{A_t} + x_t x_t^\top$
  $b_{A_t} \leftarrow b_{A_t} + r_t x_t$

Algorithm 2: LinTS (Gaussian posterior sampling)

Input: K, d, posterior scale v, ridge λ.
Init as LinUCB.
for $t = 1, 2, \dots$ **do**
  observe x_t
  for $a \in [K]$ **do**
    sample $\tilde{\theta}_a \sim \mathcal{N}(A_a^{-1} b_a,\ v^2 A_a^{-1})$
    $p_a \leftarrow \tilde{\theta}_a^\top x_t$
  pick $A_t = \arg\max_a p_a$, observe r_t, update A_{A_t}, b_{A_t} as in LinUCB.

Algorithm 3: Doubly Robust OPE

Input: log $\mathcal{D}$, target π, reward model $\hat{q}$.
for $(x_i, a_i, r_i, p_i) \in \mathcal{D}$ **do**
  $w_i \leftarrow \pi(a_i \mid x_i) \,/\, p_i$
  $v_i \leftarrow \sum_a \pi(a \mid x_i) \hat{q}(x_i, a) + w_i (r_i - \hat{q}(x_i, a_i))$
return $\frac{1}{n} \sum_i v_i$.

Example 4.0.1 (Worked numerical example)**.** Suppose $K = 2$, logger $\pi_b = (0.7, 0.3)$ (context-free for clarity), target $\pi = (0.2, 0.8)$. A log of $n = 4$ samples:

i	a_i	r_i	p_i	w_i	$w_i r_i$
1	1	1	0.7	0.286	0.286
2	2	0	0.3	2.667	0.000
3	2	1	0.3	2.667	2.667
4	1	0	0.7	0.286	0.000

$\hat{V}_{\text{IPS}} = (0.286 + 0 + 2.667 + 0)/4 = 0.738$. The single sample $i = 3$ with $w_i = 2.67$ dominates — this is the classic IPS variance explosion when the logger places little mass on the target's preferred action.

Complexity. LinUCB per round: $O(Kd^2)$ for the score loop plus $O(d^2)$ for the rank-1 update (Sherman–Morrison gives this in closed form if you maintain A_a^{-1}). LinTS adds a $O(d^3)$ Cholesky per arm per round; cache it.

Common pitfall

Propensity *overfitting.* If you re-fit π_b from logs "to recover the propensities", you bias every downstream estimator. Always log the randomised choice and its probability *at decision time.*

5 Implementation walkthrough

The mini-project lives in `project/`. The environment is `SyntheticLeadBandit`: $K = 6$ brokers, $d = 8$ lead features, hidden linear preferences with broker-specific feature specialisation and a quality gradient (broker 0 best on average, broker 5 worst).

```
class SyntheticLeadBandit:
    d: int = 8
    K: int = 6
    # theta[a] in R^d encodes broker a's hidden preferences
    def mean_reward(self, x, a):
        return sigmoid(x @ self.theta[a])
    def step(self, x, a):
        return rng.binomial(1, self.mean_reward(x, a))
```

LinUCB is one screen of code:

```
class LinUCB:
    def __init__(self, K, d, alpha=1.0, lam=1.0):
        self.A = [lam*np.eye(d) for _ in range(K)]
        self.b = [np.zeros(d)    for _ in range(K)]
        self.alpha = alpha
    def act(self, x):
        scores = []
        for a in range(self.K):
            Ainv_x = np.linalg.solve(self.A[a], x)
            mu = self.b[a] @ Ainv_x
            ucb = self.alpha * np.sqrt(max(x @ Ainv_x, 0))
            scores.append(mu + ucb)
        return int(np.argmax(scores))
    def update(self, x, a, r):
        self.A[a] += np.outer(x, x); self.b[a] += r * x
```

The offline logger is ε-greedy on its own ridge model, and *records the propensity at decision time*:

```
def act(self, x):
    p = self.propensities(x)
    a = rng.choice(self.K, p=p)
    return a, p[a]   # propensity is observed, not re-estimated
```

Doubly Robust is six lines:

```
def doubly_robust(log, pi, qhat):
    s = 0.0
    for x, a, r, pb in log:
```

```
        pix = pi(x)
        baseline = pix @ qhat(x)
        w = pix[a] / max(pb, 1e-12)
        s += baseline + w * (r - qhat(x)[a])
    return s / len(log)
```

The full code is in `project/agents.py`, `project/ope.py`, `project/train.py`. The reference `train()` function (target: LinTS regret $< 0.6T$ at $T = 2000$) runs in about 10 seconds on CPU.

6 Production pipeline: synthetic insurance in detail

This chapter walks through the engineering of the synthetic insurance lead-broker allocation pipeline at the level of detail an SRE / ML-engineer can build against. The cascade has the same structure as the ATS pipeline in Week 12 (retrieval → rerank → explore → audit → calibrate), but with degenerate retrieval (the broker panel is small).

6.1 Data contract

Every lead arriving at the allocator carries a typed payload:

- `lead_id`: a UUID for log joining.
- `features`: postcode band, vehicle group, prior-claims count, channel (direct vs. aggregator), age band, NCD years, quote channel, marketing source. Each is a categorical or numeric field with a documented type.
- `ingest_timestamp`: when the lead arrived; needed for delay-distribution modelling.
- `deduplication_hash`: catch leads submitted multiple times across channels; treat duplicates carefully (do not log multiple allocations).

6.2 Allocation decision

The allocator runs LinTS over the panel of K brokers. The decision step:

1. Compute $\tilde{\theta}_a \sim \mathcal{N}(\hat{\theta}_a, v^2 A_a^{-1})$ per broker.
2. Compute $\tilde{s}_a = x^\top \tilde{\theta}_a$ per broker.
3. Apply hard constraints: drop brokers who are offline, at quota, or have rejected this lead category before.
4. Apply exposure budgets (regulator-facing fairness floor).
5. Sample one broker proportional to $\text{softmax}(\tilde{s}_a/\tau)$; or take argmax with ε-greedy exploration.

The propensity $\pi_b(a \mid x)$ is computed at decision time as a side-effect of the sampling step — the production logger never re-estimates propensities after the fact.

6.3 Logging

The proposed decision log records context and action fields $(x, a, \pi_b(a \mid x))$, posterior diagnostics $(\tilde{\theta}_a, v, A_a^{-1})$, and version/timestamp fields `policy_version`, `model_hash`, `ingest_ts`, and `decision_ts`. A Kafka topic partitioned by hour is one storage option. The diagnostic fields ($\tilde{\theta}_a$, v, A_a^{-1}) are needed only for forensics; the critical fields for OPE are $(x, a, r, \pi_b(a \mid x),$ policy_version$)$.

6.4 Delayed outcome

A lead either binds (within 1–90 days) or expires. The reward r is right-censored: at evaluation time, some leads' fate is still unknown. The logger writes a partial record at decision time; an outcome-merger service writes the binding event (or expiry) when it arrives, joined on `lead_id`.

For OPE, the standard approach is to censor the log at a cooling-off window of $\geq$ median time-to-bind; the censoring introduces bias on the recent-data tail. The 2014 Chapelle model fits a survival head jointly with the binding model; the survival-adjusted expectation is unbiased under the survival assumption.

6.5 Audit

- *Daily*: propensity-overlap monitor (ESS / n on the OPE replay), calibration-drift monitor (sliding-window ECE).
- *Weekly*: DR-OPE on the next-candidate policy.
- *Monthly*: fairness audit (broker exposure by lead category; the synthetic insurance version of demographic-parity is exposure-by-broker).
- *Quarterly*: delayed-feedback model recalibration (Chapelle survival head).

6.6 Kill-switch

The production policy is gated behind a circuit breaker: if the live bind rate drops below 80% of the previous week's average, the allocator reverts to the previous policy version within seconds. The kill-switch prevents a bad LinTS posterior from cascading into a revenue incident.

7 Empirical results

We train LinUCB and LinTS for $T = 2000$ rounds on the synthetic insurance bandit (Figure 7.1). Both achieve sub-linear regret consistent with the $\tilde{O}(d\sqrt{T})$ theory.

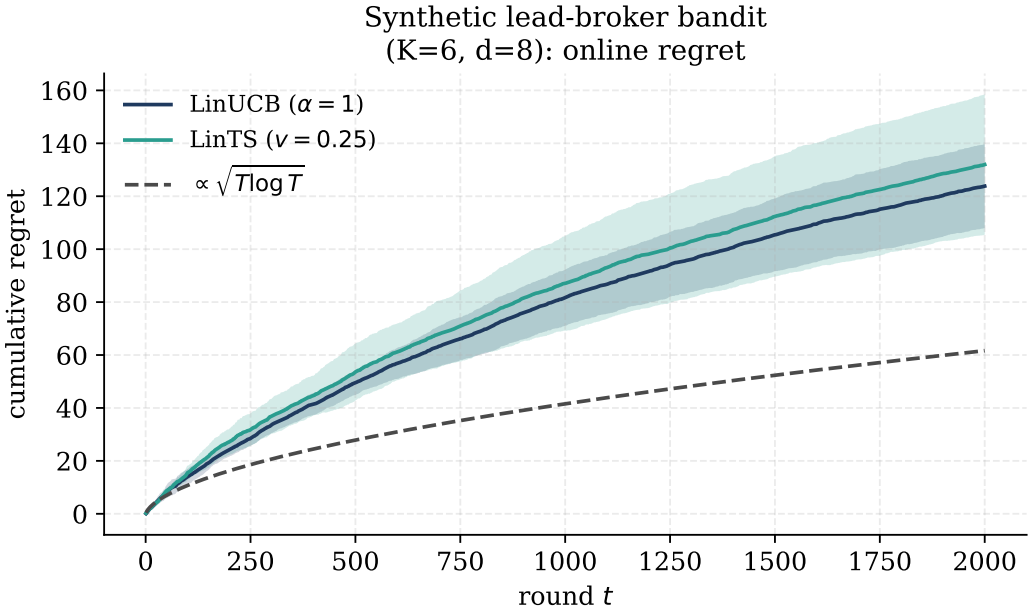


Figure 7.1: Cumulative regret on the synthetic insurance lead-broker bandit (4 seeds, mean $\pm$ std). The dashed reference is $\propto \sqrt{T \log T}$ shape; both LinUCB and LinTS track it. Random allocation would give a straight line.

Figure 7.2 compares OPE estimators on the softmax target policy. IPS has high variance (wide error bar); SNIPS is biased low (the self-normalisation deflates the estimate when weights are concentrated); DR sits closest to truth with the smallest variance.

Figure 7.3 shows the importance-weight distribution. Under the oracle-greedy target, a non-trivial fraction of weights exceed $K/\varepsilon \approx 20$ — the source of IPS variance.

Figure 7.4 shows SWITCH-DR's bias–variance trade-off. Small τ biases the estimator toward the direct method (low variance, possibly biased); large τ recovers pure DR (unbiased, high variance).

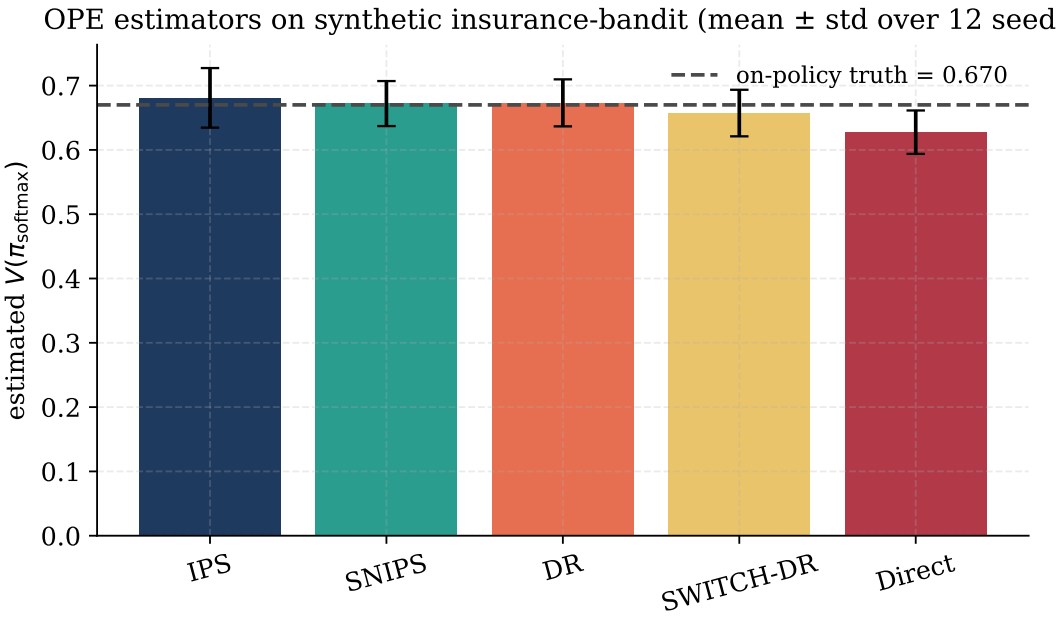


Figure 7.2: OPE estimators for a softmax target on the synthetic insurance bandit (12 seeds). Dashed line is the on-policy ground-truth value computed by Monte Carlo on fresh contexts.

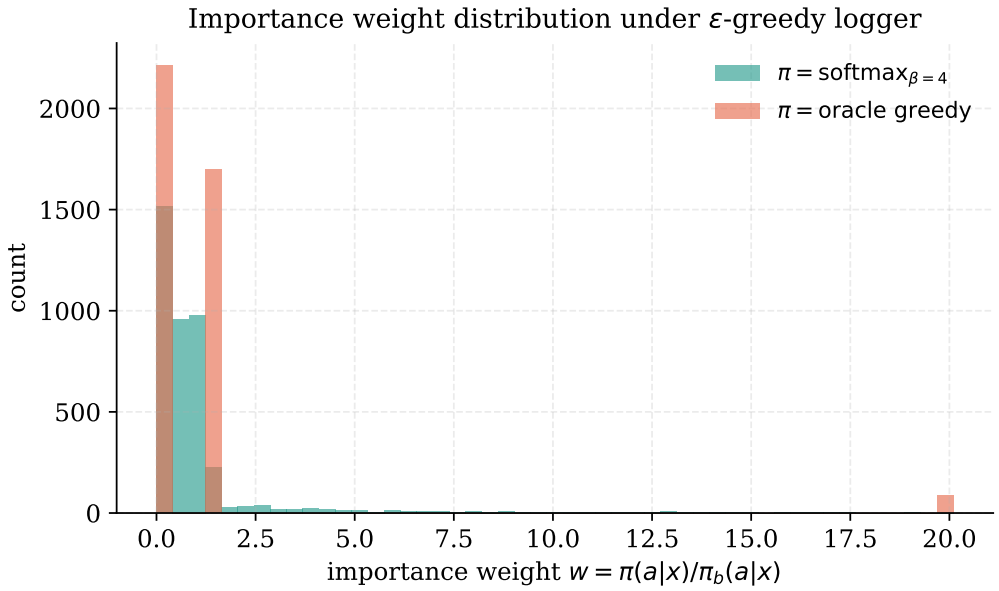


Figure 7.3: Importance-weight distribution. The peaky oracle policy puts mass where π_b has ε/K mass, producing weights of order 20.

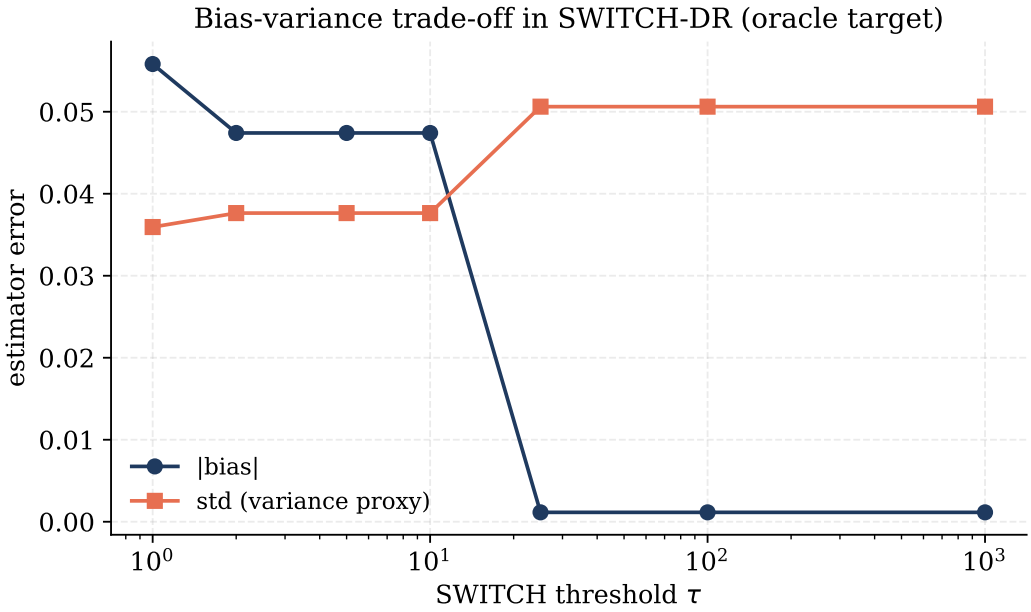


Figure 7.4: SWITCH-DR bias and variance vs. threshold τ.

8 Safety angle

Safety angle

Insurance lead routing is a *high-stakes, delayed-feedback* domain. Concrete failure modes:
F1 — Naive ε-greedy on real money. If a broker specialises in motor and you send them 30% home-insurance leads "for exploration", you destroy the broker's conversion rate, they raise their referral fees, and your unit economics collapse. *Mitigation:* cap ε per arm using *Conservative Bandits* (Wu, Srikant et al., NeurIPS 2016): only explore if a safety constraint $\hat{V}(\pi) \geq (1-\zeta)\hat{V}(\pi_{\text{safe}})$ holds with high probability.
F2 — IPS with missing propensities. A common production sin: the production policy is changed by an engineer, the propensity logger is not updated, and OPE silently becomes wrong. *Mitigation:* log policy version hashes alongside the propensity; reject OPE jobs that span a version boundary.
F3 — Delayed feedback bias. A lead binds in 1–90 days. If you evaluate today, recent leads are systematically under-credited. The estimator is biased *against* new policies (which only have recent data). *Mitigation:* censor logs to a cooling-off window of at least the median time-to-bind; or fit a survival model and use predicted-survival adjusted rewards (Chapelle 2014; Vernade et al. 2017).
F4 — Overlap (positivity) failures. A new broker (cold-start) has zero historical mass on most leads. Any target policy that prefers them is un-evaluable by IPS / DR. *Mitigation:* hierarchical Thompson priors (Kveton et al. 2021); explicit cold-start exploration budgets; honest reporting of ESS (effective sample size).
F5 — Propensity overfitting. Re-fitting π_b from logs leaks the reward into the propensity, biasing IPS. *Mitigation:* log propensities at decision time.
Two cites: Bottou et al., *Counterfactual Reasoning and Learning Systems*, JMLR 2013; Joachims, Swaminathan, de Rijke, *Deep Learning with Logged Bandit Feedback*, ICLR 2018 (BanditNet).

9 Matching application

Matching application

The synthetic insurance lead-broker problem is the canonical applied bandit. The mapping:

Bandit primitive	synthetic insurance analogue
context x	lead features (postcode, vehicle, age, channel, prior claims)
arm a	broker (one of K panel members)
reward r	bind / no-bind (binary)
propensity $\pi_b(a\|x)$	probability the production allocator chose broker a for lead x
target π	candidate policy (re-weighted LinTS, or a fairness-constrained variant)
delayed feedback	1–90 days from quote to bind

Engineering implications:

1. Log $(x, a, r, p,$ policy_version_hash$)$ *at decision time*.
2. Train a calibrated $\hat{q}(x, a)$ (Beta-calibration on a holdout split).
3. Run weekly DR + SWITCH evaluations against the next-candidate policy.
4. Gate deploys behind a conservative bandit bound (Wu et al. 2016).
5. Run a delayed-feedback adjustment (survival head, Chapelle 2014).

A similar mapping works for ATS candidate–job ranking (Week 12), but position bias and slate structure add a layer of correction.

10 Exercises

See `workbook.ipynb` for full statements, hints, and graded tests. Summary:

- **E1** ($\star$) Compute the UCB1 bonus for $t = 100$, $n_a = 10$.
- **E2** ($\star$) Implement the KL-UCB upper-confidence solver.
- **E3** ($\star\star$) Bernoulli Thompson Sampling on a 3-armed bandit; confirm $O(\log T)$ regret growth empirically.
- **E4** ($\star\star$) Train LinUCB on the synthetic insurance bandit; report cumulative regret.
- **E5** ($\star\star$) Collect ε-greedy logs; compute IPS and SNIPS for a uniform target.
- **E6** ($\star\star$) Doubly Robust against the softmax target; compare to the on-policy truth.
- **E7** ($\star$) Effective sample size of importance weights.
- **E8** ($\star\star$) SWITCH-DR; vary τ; observe bias/variance.
- **E9** ($\star\star\star$, *open-ended*, *synthetic insurance objective*) For three counterfactual policies (uniform, softmax-true, oracle-greedy) compare SNIPS, DR, and SWITCH-DR estimates against on-policy ground truth. Write a one-page recommendation memo to the synthetic insurance allocation team.

11 Further reading

The `extended_reading.pdf` document contains annotated summaries of five foundational papers: Auer et al. 2002 (UCB1), Agrawal & Goyal 2013 (LinTS), Li et al. 2010 (LinUCB), Dudík et al. 2011 (DR), Joachims et al. 2018 (BanditNet). For broader context, Lattimore & Szepesvári, *Bandit Algorithms* (2020), is the textbook; Bietti, Agarwal, Langford, *A Contextual Bandit Bake-off* (JMLR 2021) is the empirical state of the art; and Saito & Joachims (ICML 2022) introduce MIPS for large-action OPE — the natural next step for synthetic insurance / ATS when K grows from 6 to 6000.

Sourced from `research/bandits_matching_sota.md` sections 1–3 (stochastic bandits, contextual bandits, OPE) and `research/safety_rl_sota.md` (conservative bandits, batch RL safety).